Estructuras de Datos II

Complejidad, Ordenamiento y Diseño Algorítmico

Análisis Completo en Java

Notación Asintótica · Análisis Iterativo y Recursivo · Anuladores · Ordenamiento · Backtracking · Programación Dinámica · Voraces · Divide y Vencerás

Lenguaje	Java
Matemáticas	Sumatorias, recurrencias, anuladores
Nivel	Intermedio – Avanzado

Este documento cubre el análisis de complejidad computacional (iterativa con sumatorias, recursiva con el método de los anuladores), los principales algoritmos de ordenamiento con su análisis formal, y las técnicas de diseño algorítmico: backtracking, programación dinámica, algoritmos voraces y divide y vencerás, con sus estructuras de datos auxiliares explicadas desde cero.

Contents

1 Complejidad Computacional: Notación Asintótica **2**
1.1 Reglas de simplificación . . . 3

2 Análisis de Complejidad Iterativa **3**
2.1 Sumatorias fundamentales . . . 3
2.2 Análisis de bucles simples . . . 4
2.3 Análisis de bucles anidados . . . 4
2.4 Resumen: cómo leer el patrón de un bucle . . . 6

3 Análisis de Complejidad Recursiva: Método de los Anuladores **6**
3.1 Ecuaciones de recurrencia . . . 6
3.2 El operador de avance E y los anuladores . . . 6
3.3 Procedimiento paso a paso . . . 7
3.4 Teorema Maestro (referencia rápida) . . . 9

4 Algoritmos de Ordenamiento **9**
4.1 Bubble Sort . . . 10
4.2 Insertion Sort . . . 10
4.3 Selection Sort . . . 11
4.4 Counting Sort . . . 12
4.5 Radix Sort . . . 13
4.6 MergeSort . . . 13
4.7 QuickSort . . . 15

5 Técnicas de Diseño Algorítmico **16**
5.1 Divide y Vencerás . . . 16
5.2 Backtracking y Branch & Bound (Pruning) . . . 17
5.2.1 Estructura de datos auxiliar: Árbol de Decisión . . . 17
5.3 Programación Dinámica . . . 20
5.3.1 Estructura de datos auxiliar: Tabla de Memoización . . . 20
5.4 Algoritmos Voraces (*Greedy*) . . . 23
5.5 Comparativa de Técnicas . . . 27

6 Resumen Integral y Guía de Decisión **27**

7 Conclusiones **28**

1 Complejidad Computacional: Notación Asintótica

La **complejidad computacional** mide cómo crece el tiempo de ejecución (o el uso de memoria) de un algoritmo en función del tamaño de la entrada n. No nos interesa el tiempo exacto en segundos, sino la *tasa de crecimiento*.

Notaciones asintóticas fundamentales

Sean $f(n)$ y $g(n)$ funciones de los naturales a los reales positivos:

- $f(n) = O(g(n))$ (**cota superior**): existen $c > 0$ y n_0 tales que $f(n) \leq c \cdot g(n)$ para todo $n \geq n_0$.
  "f no crece más rápido que g".
- $f(n) = \Omega(g(n))$ (**cota inferior**): existen $c > 0$ y n_0 tales que $f(n) \geq c \cdot g(n)$ para todo $n \geq n_0$.
  "f crece al menos tan rápido como g".
- $f(n) = \Theta(g(n))$ (**cota ajustada**): $f = O(g)$ y $f = \Omega(g)$ simultáneamente.
  "f y g crecen exactamente igual de rápido".

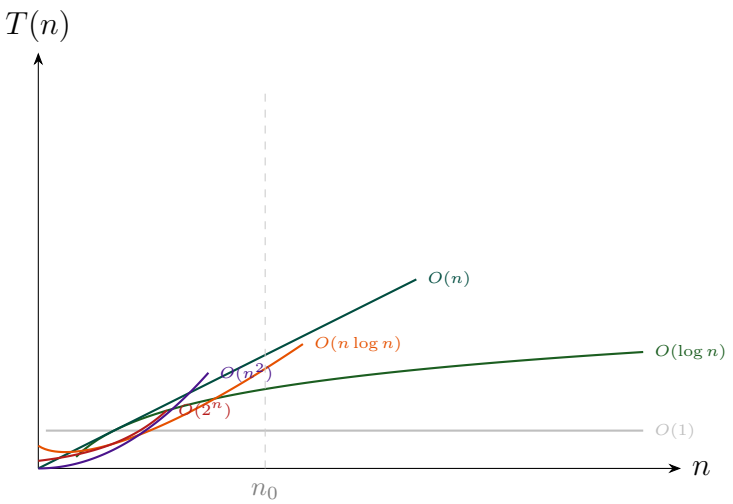

Figure 1: Clases de complejidad ordenadas de menor a mayor crecimiento.

Table 1: Clases de complejidad comunes

Notación	Nombre	Ejemplo	$n = 10^6$
$O(1)$	Constante	Acceso a arreglo por índice	1 op.
$O(\log n)$	Logarítmico	Búsqueda binaria	20 ops.
$O(n)$	Lineal	Recorrido de lista	10^6 ops.
$O(n \log n)$	Linealog.	MergeSort, QuickSort prom.	2×10^7 ops.
$O(n^2)$	Cuadrático	BubbleSort, selección	10^{12} ops.
$O(n^3)$	Cúbico	Multiplicación de matrices	10^{18} ops.
$O(2^n)$	Exponencial	Subconjuntos, backtracking	Imposible
$O(n!)$	Factorial	Permutaciones brutas	Imposible

1.1 Reglas de simplificación

Propiedades clave de la notación O

1. **Constantes**: $O(c \cdot f(n)) = O(f(n))$ ej. $O(5n^2) = O(n^2)$
2. **Suma**: $O(f(n) + g(n)) = O(\max(f(n), g(n)))$ ej. $O(n^2 + n) = O(n^2)$
3. **Producto**: $O(f(n) \cdot g(n)) = O(f(n)) \cdot O(g(n))$
4. **Transitividad**: si $f = O(g)$ y $g = O(h)$ entonces $f = O(h)$

2 Análisis de Complejidad Iterativa

El análisis iterativo consiste en contar exactamente cuántas operaciones elementales ejecuta el algoritmo, expresarlo como una función $T(n)$ mediante **sumatorias**, y luego simplificar con la notación asintótica.

2.1 Sumatorias fundamentales

Las siguientes fórmulas son esenciales para el análisis:

Sumatorias cerradas

$$\sum_{i=1}^{n} 1 = n \tag{1}$$

$$\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \Theta(n^2) \tag{2}$$

$$\sum_{i=1}^{n} i^2 = \frac{n(n+1)(2n+1)}{6} = \Theta(n^3) \tag{3}$$

$$\sum_{i=0}^{n} r^i = \frac{r^{n+1}-1}{r-1} \quad (r \neq 1) \tag{4}$$

$$\sum_{i=1}^{n} \log i = \log(n!) = \Theta(n \log n) \tag{5}$$

2.2 Análisis de bucles simples

Ejemplo 1. *Analizar la complejidad del siguiente fragmento:*

Listing 1: Bucle simple $O(n)$

```
int suma = 0;
for (int i = 0; i < n; i++) {    // ejecuta n veces
    suma += i;                    // 1 operacion por iteracion
}
```

$T(n) = \sum_{i=0}^{n-1} 1 = n \Rightarrow T(n) = \Theta(n)$

Ejemplo 2. *Analizar un bucle con variable que se duplica:*

Listing 2: Bucle logarítmico $O(\log n)$

```
int i = 1;
while (i < n) {    // i toma valores: 1, 2, 4, 8, ..., 2^k
    i = i * 2;     // duplicamos en cada paso
}
```

El bucle se ejecuta mientras $2^k < n$, *es decir* $k < \log_2 n$.
$T(n) = \lfloor \log_2 n \rfloor \Rightarrow T(n) = \Theta(\log n)$

2.3 Análisis de bucles anidados

Ejemplo 3. *Dos bucles anidados cuadráticos:*

Listing 3: Bucles anidados $O(n^2)$

```java
for (int i = 0; i < n; i++) {            // n iteraciones
    for (int j = 0; j < n; j++) {        // n iteraciones cada vez
        System.out.println(i + " " + j); // 1 operacion
    }
}
```

$$T(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} 1 = \sum_{i=0}^{n-1} n = n \cdot n = n^2 \Rightarrow T(n) = \Theta(n^2)$$

Ejemplo 4. *Bucles anidados con límite dependiente: análisis del BubbleSort.*

Listing 4: Bucle triangular $O(n^2)$

```java
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n - i - 1; j++) { // limite decrece con i
        // comparacion e intercambio
    }
}
```

$$T(n) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-i-2} 1 = \sum_{i=0}^{n-1} (n-i-1) = (n-1)+(n-2)+\cdots+1+0 = \frac{n(n-1)}{2} = \Theta(n^2)$$

Usando la fórmula (2): $\displaystyle\sum_{k=0}^{n-1} k = \frac{n(n-1)}{2}$.

Ejemplo 5. *Bucle con crecimiento logarítmico en el interno:*

Listing 5: $O(n \log n)$

```java
for (int i = 1; i <= n; i++) {     // n iteraciones externas
    for (int j = 1; j <= n; j *= 2) { // log(n) internas
        // operacion
    }
}
```

$$T(n) = \sum_{i=1}^{n} \lfloor \log_2 n \rfloor = n \cdot \log_2 n \Rightarrow T(n) = \Theta(n \log n)$$

2.4 Resumen: cómo leer el patrón de un bucle

Table 2: Patrones de bucles y su complejidad

Patrón de iteración	Ejemplo de control	Complejidad
Incremento constante	`i++`	$O(n)$
Decremento constante	`i-`	$O(n)$
Multiplicación por r	`i *= 2`	$O(\log n)$
Raíz cuadrada	`i = (int)sqrt(i)`	$O(\log \log n)$
Bucle en bucle (igual)	`i++` en `j++`	$O(n^2)$
Bucle triangular	`j < n-i`	$O(n^2)$
Externo lineal, interno log	`j *= 2`	$O(n \log n)$

3 Análisis de Complejidad Recursiva: Método de los Anuladores

Cuando un algoritmo se llama a sí mismo, su tiempo se expresa como una **ecuación de recurrencia**. El **método de los anuladores** (u operadores de diferencia) resuelve estas ecuaciones de forma algebraica.

3.1 Ecuaciones de recurrencia

Forma general de una recurrencia

$$T(n) = a \cdot T(n-b) + f(n), \qquad T(0) = c_0$$

$$T(n) = a \cdot T(n/b) + f(n), \qquad T(1) = c_0$$

donde a = número de llamadas recursivas, b = reducción del problema, $f(n)$ = trabajo en cada nivel (parte no recursiva).

3.2 El operador de avance E y los anuladores

El método opera sobre secuencias. Define el **operador de avance**:

$$E[T(n)] = T(n+1) \qquad \Rightarrow \qquad E^k[T(n)] = T(n+k)$$

Un **anulador** de una secuencia $f(n)$ es un polinomio en E tal que aplicado a $f(n)$ da cero. Los anuladores fundamentales son:

Table 3: Tabla de anuladores fundamentales

Secuencia $f(n)$	Anulador	Solución general
c (constante)	$(E-1)$	$A \cdot 1^n = A$
n	$(E-1)^2$	$A + Bn$
n^2	$(E-1)^3$	$A + Bn + Cn^2$
n^k	$(E-1)^{k+1}$	Polinomio de grado k
a^n	$(E-a)$	$A \cdot a^n$
$n \cdot a^n$	$(E-a)^2$	$(A + Bn) \cdot a^n$
$n^k \cdot a^n$	$(E-a)^{k+1}$	Polinomio$\times a^n$
$\sin(cn)$, $\cos(cn)$	$E^2 - 2\cos(c)E + 1$	Combinación trigonom.

3.3 Procedimiento paso a paso

Pasos del método de los anuladores

1. Escribir la recurrencia en la forma $T(n) - a\,T(n-1) = f(n)$
2. Reescribir con el operador E: $(E^k - a)\,T = f(n)$ (usando $T(n) = E^k T(n-k)$)
3. Encontrar el anulador $A(E)$ de $f(n)$
4. Multiplicar ambos lados: $A(E) \cdot (E^k - a)\,T = 0$
5. La solución general es la suma de las soluciones correspondientes a cada raíz del polinomio combinado
6. Aplicar condiciones iniciales para hallar las constantes

Ejemplo 6. *Resolver* $T(n) = T(n-1) + 1$, $T(0) = 0$.

Paso 1: *Reescribir:* $T(n) - T(n-1) = 1$

Paso 2: *Con operador* E*:* $(E-1)\,T(n-1) = 1$, *es decir* $(E-1)\,T = E \cdot 1 \to$ *en notación directa:* $(E-1)T(n) = 1$.

Paso 3: *El anulador de* 1 *(constante) es* $(E-1)$.

Paso 4: *Aplicar anulador:* $(E-1)(E-1)\,T = 0 \Rightarrow (E-1)^2 T = 0$.

Paso 5: *Raíz doble* $E = 1$*: solución general* $T(n) = A + Bn$.

Paso 6: $T(0) = 0 \Rightarrow A = 0$; $T(1) = 1 \Rightarrow B = 1$.

$$\boxed{T(n) = n = \Theta(n)}$$

Ejemplo 7. *Resolver* $T(n) = 2\,T(n-1) + 1$, $T(0) = 1$.

Paso 1–2: $(E-2)\,T(n) = 1$ *(después de reordenar).*

Paso 3: *Anulador de* 1: $(E-1)$.

Paso 4: $(E-1)(E-2)\,T = 0$.

Paso 5: *Raíces* $E = 1$ *y* $E = 2$: $T(n) = A \cdot 1^n + B \cdot 2^n = A + B \cdot 2^n$.

Paso 6: $T(0) = 1 \Rightarrow A + B = 1$; $T(1) = 3 \Rightarrow A + 2B = 3 \Rightarrow B = 2,\ A = -1$.

$$\boxed{T(n) = -1 + 2^{n+1} = \Theta(2^n)}$$

Ejemplo 8. *Resolver* $T(n) = 2\,T(n/2) + n$ *(MergeSort),* $T(1) = 1$.

Se hace el cambio de variable $n = 2^k$ $(k = \log_2 n)$, *definiendo* $S(k) = T(2^k)$:

$$S(k) = 2\,S(k-1) + 2^k, \qquad S(0) = 1$$

Paso 2: $(E-2)\,S = 2^k$.

Paso 3: *Anulador de* 2^k: $(E-2)$.

Paso 4: $(E-2)^2\,S = 0$.

Paso 5: *Raíz doble* $E = 2$: $S(k) = (A + Bk) \cdot 2^k$.

Paso 6: $S(0) = 1 \Rightarrow A = 1$; $S(1) = 3 \Rightarrow 2(1+B) = 3 \Rightarrow B = \frac{1}{2}$.

Volviendo a n: $T(n) = S(\log n) = (1 + \frac{1}{2}\log_2 n) \cdot n$.

$$\boxed{T(n) = n + \tfrac{n}{2}\log_2 n = \Theta(n \log n)}$$

Ejemplo 9. *Resolver* $T(n) = T(n-1) + n$, $T(0) = 0$.

$(E-1)\,T = n$. *Anulador de* n: $(E-1)^2$.

$(E-1)^3\,T = 0$. *Raíz triple:* $T(n) = A + Bn + Cn^2$.

$T(0) = 0 \Rightarrow A = 0$; $T(1) = 1 \Rightarrow B + C = 1$; $T(2) = 3 \Rightarrow 2B + 4C = 3 \Rightarrow C = \frac{1}{2},\ B = \frac{1}{2}$.

$$\boxed{T(n) = \frac{n}{2} + \frac{n^2}{2} = \frac{n(n+1)}{2} = \Theta(n^2)}$$

3.4 Teorema Maestro (referencia rápida)

Para recurrencias de la forma $T(n) = a\,T(n/b) + f(n)$:

Teorema Maestro

Sea $p = \log_b a$. Entonces:

- Si $f(n) = O(n^{p-\varepsilon})$ para algún $\varepsilon > 0$: $T(n) = \Theta(n^p)$
- Si $f(n) = \Theta(n^p)$: $T(n) = \Theta(n^p \log n)$
- Si $f(n) = \Omega(n^{p+\varepsilon})$ y se cumple la condición de regularidad: $T(n) = \Theta(f(n))$

Table 4: Aplicaciones del Teorema Maestro

Recurrencia	a	b	$f(n)$	Resultado
MergeSort	2	2	n	$\Theta(n \log n)$
Búsq. binaria	1	2	1	$\Theta(\log n)$
Strassen	7	2	n^2	$\Theta(n^{2.81})$
QuickSort prom.	2	2	n	$\Theta(n \log n)$

4 Algoritmos de Ordenamiento

Table 5: Comparativa de algoritmos de ordenamiento

Algoritmo	Mejor	Prom.	Peor	Mem.	Estable	In-place
Bubble	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí	Sí
Insertion	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sí	Sí
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No	Sí
Counting	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(k)$	Sí	No
Radix	$O(nk)$	$O(nk)$	$O(nk)$	$O(n+k)$	Sí	No
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sí	No
QuickSort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	Sí

4.1 Bubble Sort

Compara pares adyacentes y los intercambia si están en orden incorrecto. El elemento más grande "burbujea" al final en cada pasada.

Inicial:	5	3	8	1	9	2
Pasada 1:	3	5	1	8	2	9
Pasada 2:	3	1	5	2	8	9
Final:	1	2	3	5	8	9

Verde = ya ordenado y fijo en su posición

Figure 2: Bubble Sort: los elementos grandes "burbujean" al final.

Listing 6: Bubble Sort con optimización de parada temprana

```java
public static void bubbleSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        boolean intercambio = false;
        // cada pasada lleva el maximo al final
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int tmp = arr[j]; arr[j] = arr[j+1]; arr[j+1]
                    = tmp;
                intercambio = true;
            }
        }
        // Si no hubo intercambio, el arreglo ya esta ordenado
        if (!intercambio) break;  // optimizacion: O(n) en
            mejor caso
    }
}
// T(n) = sum_{i=0}^{n-2} (n-i-1) = n(n-1)/2 = O(n^2)
```

4.2 Insertion Sort

Construye el arreglo ordenado elemento por elemento, insertando cada nuevo elemento en su posición correcta dentro de la parte ya ordenada.

Listing 7: Insertion Sort

```java
public static void insertionSort(int[] arr) {
    for (int i = 1; i < arr.length; i++) {
        int clave = arr[i];       // elemento a insertar
        int j     = i - 1;
        // desplazar hacia la derecha los elementos mayores que clave
        while (j >= 0 && arr[j] > clave) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = clave;      // insertar en la posicion correcta
    }
}
// Mejor caso (ya ordenado): T(n) = n-1 = O(n)
// Peor caso (invertido):    T(n) = sum_{i=1}^{n-1} i = n(n-1)/2 = O(n^2)
```

4.3 Selection Sort

En cada pasada busca el mínimo del subarreglo no ordenado y lo intercambia con el primer elemento de ese subarreglo.

Listing 8: Selection Sort

```java
public static void selectionSort(int[] arr) {
    int n = arr.length;
    for (int i = 0; i < n - 1; i++) {
        int minIdx = i;
        // buscar el minimo en arr[i+1 ... n-1]
        for (int j = i + 1; j < n; j++)
            if (arr[j] < arr[minIdx]) minIdx = j;

        // intercambiar con la posicion i
        int tmp = arr[i]; arr[i] = arr[minIdx]; arr[minIdx] = tmp;
    }
}
// T(n) = sum_{i=0}^{n-2}(n-i-1) = n(n-1)/2 = O(n^2) siempre
// Ventaja: exactamente n-1 intercambios (minimo posible)
```

4.4 Counting Sort

Algoritmo **no comparativo**: cuenta las ocurrencias de cada valor y usa esas cuentas para posicionar directamente cada elemento. Requiere que los valores sean enteros en un rango $[0, k]$ conocido.

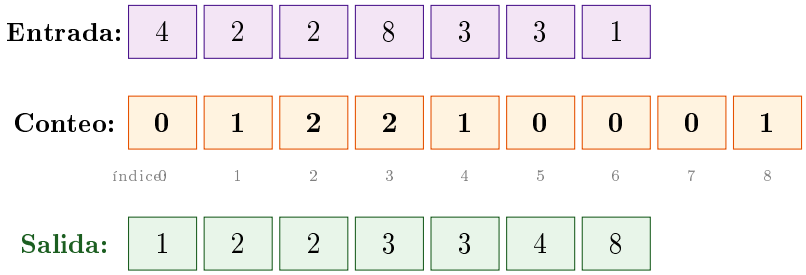

Figure 3: Counting Sort: contar ocurrencias, luego reconstruir.

Listing 9: Counting Sort

```java
public static void countingSort(int[] arr, int k) {
    // k = valor maximo en arr
    int[] conteo = new int[k + 1];      // O(k) memoria

    // Paso 1: contar ocurrencias
    for (int val : arr) conteo[val]++;

    // Paso 2: acumular (prefix sum) -> posicion de salida
    for (int i = 1; i <= k; i++) conteo[i] += conteo[i - 1];

    // Paso 3: construir arreglo de salida (recorrido inverso
        = estable)
    int[] salida = new int[arr.length];
    for (int i = arr.length - 1; i >= 0; i--) {
        salida[--conteo[arr[i]]] = arr[i];
    }

    // Copiar resultado
    System.arraycopy(salida, 0, arr, 0, arr.length);
}
// T(n,k) = O(n + k)    Optimo cuando k = O(n)
// Limitacion: solo funciona con enteros no negativos
```

4.5 Radix Sort

Extiende Counting Sort para números grandes: ordena dígito a dígito (de menos significativo a más significativo), usando un algoritmo estable (normalmente Counting Sort) en cada pasada.

Listing 10: Radix Sort LSD (dígito menos significativo primero)

```java
public static void radixSort(int[] arr) {
    int max = Arrays.stream(arr).max().getAsInt();

    // Ordenar por cada digito: 1, 10, 100, ...
    for (int exp = 1; max / exp > 0; exp *= 10)
        countingSortPorDigito(arr, exp);
}

// Counting sort estable usando el digito en posicion 'exp'
private static void countingSortPorDigito(int[] arr, int exp)
    {
    int n = arr.length;
    int[] salida = new int[n];
    int[] conteo = new int[10];    // digitos 0-9

    for (int v : arr)              conteo[(v / exp) % 10]++;
    for (int i = 1; i < 10; i++)   conteo[i] += conteo[i - 1];
    for (int i = n - 1; i >= 0; i--) {
        salida[--conteo[(arr[i] / exp) % 10]] = arr[i];
    }
    System.arraycopy(salida, 0, arr, 0, n);
}
// T(n) = O(d*(n+10)) = O(d*n) donde d = num. digitos del
    maximo
// Para enteros de 32 bits: d <= 10, por lo tanto O(n) en la
    practica
```

4.6 MergeSort

Divide el arreglo a la mitad recursivamente hasta llegar a subarreglos de tamaño 1, luego los **mezcla (merge)** en orden. Paradigma *divide y vencerás* puro.

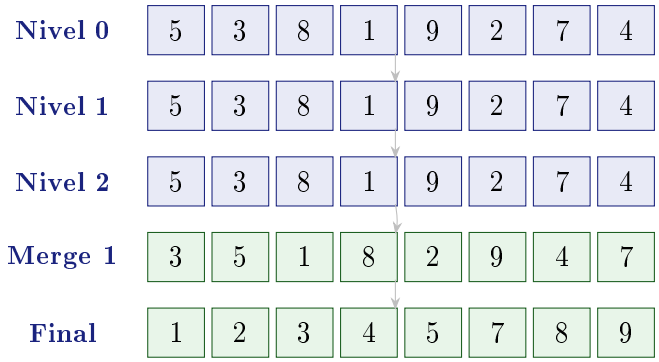

Figure 4: MergeSort: divide hasta llegar a elementos solos, luego mezcla.

Listing 11: MergeSort con análisis de recurrencia

```java
public static void mergeSort(int[] arr, int izq, int der) {
    if (izq >= der) return;            // caso base: 1 elemento
    int mid = (izq + der) / 2;
    mergeSort(arr, izq, mid);          // T(n/2)
    mergeSort(arr, mid + 1, der);      // T(n/2)
    merge(arr, izq, mid, der);         // O(n)
}
// Recurrencia: T(n) = 2T(n/2) + n => T(n) = O(n log n) por anuladores

private static void merge(int[] arr, int izq, int mid, int der) {
    // copiar subarreglos temporales
    int[] L = Arrays.copyOfRange(arr, izq, mid + 1);
    int[] R = Arrays.copyOfRange(arr, mid + 1, der + 1);
    int i = 0, j = 0, k = izq;

    // mezclar en orden
    while (i < L.length && j < R.length)
        arr[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];

    while (i < L.length) arr[k++] = L[i++];
    while (j < R.length) arr[k++] = R[j++];
}

// Uso
int[] arr = {5,3,8,1,9,2,7,4};
mergeSort(arr, 0, arr.length - 1);
// arr = {1,2,3,4,5,7,8,9}
```

4.7 QuickSort

Elige un **pivote**, particiona el arreglo de modo que los elementos menores queden a su izquierda y los mayores a su derecha, y recursa en ambas mitades.

Listing 12: QuickSort con partición de Lomuto

```java
public static void quickSort(int[] arr, int izq, int der) {
    if (izq >= der) return;
    int pivoteIdx = particionar(arr, izq, der);
    quickSort(arr, izq, pivoteIdx - 1);     // subarreglo izquierdo
    quickSort(arr, pivoteIdx + 1, der);     // subarreglo derecho
}

// Particion de Lomuto: pivote = ultimo elemento
private static int particionar(int[] arr, int izq, int der) {
    int pivote = arr[der];
    int i = izq - 1;              // indice del ultimo elemento menor

    for (int j = izq; j < der; j++) {
        if (arr[j] <= pivote) {
            i++;
            int tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
        }
    }
    // colocar pivote en su posicion final
    int tmp = arr[i+1]; arr[i+1] = arr[der]; arr[der] = tmp;
    return i + 1;
}
// Caso promedio: T(n) = 2T(n/2) + n = O(n log n)  [particion equilibrada]
// Peor caso:     T(n) = T(n-1) + n  = O(n^2)       [arreglo ya ordenado]
// Mejor practica: usar pivote aleatorio para evitar el peor caso

public static void quickSortAleatorio(int[] arr, int izq, int der) {
    if (izq >= der) return;
    // elegir pivote aleatorio e intercambiar con el ultimo
```

```
    int r = izq + (int)(Math.random() * (der - izq + 1));
    int tmp = arr[r]; arr[r] = arr[der]; arr[der] = tmp;
    int p = particionar(arr, izq, der);
    quickSortAleatorio(arr, izq, p - 1);
    quickSortAleatorio(arr, p + 1, der);
}
```

5 Técnicas de Diseño Algorítmico

Las técnicas de diseño son *estrategias generales* que se aplican a familias enteras de problemas.

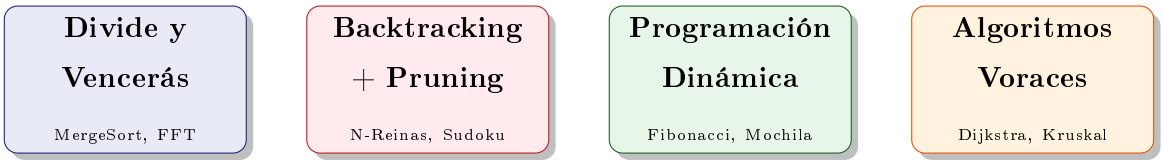

Figure 5: Las cuatro técnicas principales de diseño algorítmico.

5.1 Divide y Vencerás

Estructura: Dividir el problema en subproblemas más pequeños del mismo tipo, resolverlos recursivamente, y combinar sus soluciones.

Esquema Divide y Vencerás

1. **Dividir**: partir el problema de tamaño n en a subproblemas de tamaño n/b.
2. **Vencer**: resolver cada subproblema recursivamente.
3. **Combinar**: unir las soluciones parciales en tiempo $f(n)$.

Recurrencia: $T(n) = a\,T(n/b) + f(n)$

Listing 13: Búsqueda Binaria — Divide y Vencerás clásico

```
// T(n) = T(n/2) + 1 => por Teorema Maestro: O(log n)
public static int busquedaBinaria(int[] arr, int objetivo, int
    izq, int der) {
    if (izq > der) return -1;         // no encontrado

    int mid = (izq + der) / 2;
    if      (arr[mid] == objetivo) return mid;
    else if (arr[mid] >  objetivo) return busquedaBinaria(arr,
        objetivo, izq, mid-1);
    else                           return busquedaBinaria(arr,
        objetivo, mid+1, der);
```

```
}
```

Listing 14: Exponenciación rápida — Divide y Vencerás

```
// Calcula base^exp en O(log exp) en vez de O(exp)
// T(n) = T(n/2) + O(1) = O(log n)
public static long potencia(long base, int exp) {
    if (exp == 0) return 1;
    if (exp % 2 == 0) {
        long mitad = potencia(base, exp / 2);
        return mitad * mitad;          // reutilizar: solo 1 llamada recursiva
    } else {
        return base * potencia(base, exp - 1);
    }
}
// Comparacion:
// Potencia naive: 2^30 = 30 multiplicaciones
// Potencia rapida: 2^30 = 5 multiplicaciones (log2(30) ~ 5)
```

5.2 Backtracking y Branch & Bound (Pruning)

Backtracking explora recursivamente el espacio de soluciones como un **árbol de decisión**: en cada nodo toma una decisión, y si llega a un estado inválido, *retrocede* (deshace la decisión) y prueba otra rama.

5.2.1 Estructura de datos auxiliar: Árbol de Decisión

El árbol de decisión es la estructura conceptual subyacente del backtracking. Cada **nodo** representa un estado parcial de la solución. Cada **arista** representa elegir un valor para una variable. Las **hojas** son soluciones completas (válidas o no).

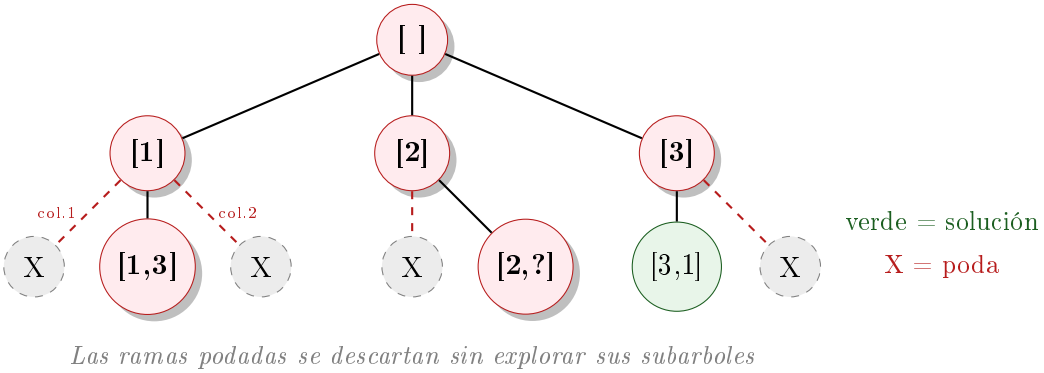

Figure 6: Árbol de decisión del problema de las N-Reinas (simplificado a 3). Las ramas en rojo son podadas por la restricción de columna.

Branch & Bound (Poda y Ramificación)

Pruning añade una función de **cota** (bound): si el costo de la solución parcial ya supera el mejor conocido, se poda la rama completamente. Transforma backtracking exponencial en algo mucho más práctico en la mayoría de instancias reales.

Listing 15: N-Reinas con backtracking y pruning

```java
public class NReinas {

    private int   n;
    private int[] reina;        // reina[fila] = columna donde esta la reina
    private int   soluciones;

    public NReinas(int n) { this.n = n; this.reina = new int[n]; }

    public void resolver() { backtrack(0); }

    private void backtrack(int fila) {
        if (fila == n) {
            soluciones++;
            imprimir();
            return;
        }

        for (int col = 0; col < n; col++) {
            if (esValido(fila, col)) {     // PRUNING: verificar antes de recursar
                reina[fila] = col;
                backtrack(fila + 1);       // explorar esta rama
                // reina[fila] se sobreescribe en la siguiente iteracion
                // (backtrack implicito: no necesitamos "deshacer" el int)
            }
        }
    }

    private boolean esValido(int fila, int col) {
```

```java
        for (int f = 0; f < fila; f++) {
            // misma columna O misma diagonal
            if (reina[f] == col ||
                Math.abs(reina[f] - col) == Math.abs(f - fila)
                    )
                return false;  // conflicto -> PODAR esta rama
        }
        return true;
    }

    private void imprimir() {
        System.out.print("Solucion " + soluciones + ": ");
        for (int r : reina) System.out.print(r + " ");
        System.out.println();
    }
}
// Complejidad sin poda: O(n!)
// Con poda: mucho menor en la practica (aprox. O(n! /
    factor_de_poda))
// n=8: 92 soluciones de 40320 posibles sin poda
```

Listing 16: Suma de subconjuntos con backtracking

```java
public class SumaSubconjuntos {

    // Encontrar todos los subconjuntos de 'nums' que sumen '
        objetivo'
    public List<List<Integer>> encontrar(int[] nums, int
        objetivo) {
        List<List<Integer>> resultado = new ArrayList<>();
        Arrays.sort(nums);  // ordenar para facilitar la poda
        backtrack(nums, objetivo, 0, new ArrayList<>(),
            resultado);
        return resultado;
    }

    private void backtrack(int[] nums, int restante, int
        inicio,
                           List<Integer> actual, List<List<
                               Integer>> res) {
        if (restante == 0) {
            res.add(new ArrayList<>(actual));
            return;
```

```java
        }

        for (int i = inicio; i < nums.length; i++) {
            // PODA: si el actual ya supera el objetivo, los siguientes
            // (que son mayores por el orden) tampoco servirán
            if (nums[i] > restante) break;

            actual.add(nums[i]);
            backtrack(nums, restante - nums[i], i + 1, actual, res);
            actual.remove(actual.size() - 1);  // DESHACER: backtrack explícito
        }
    }
}
```

5.3 Programación Dinámica

La **programación dinámica** (PD) resuelve problemas con *subestructura óptima* y *subproblemas solapados*: en lugar de recalcular los mismos subproblemas repetidamente, los **almacena en una tabla** (memoización o tabulación).

5.3.1 Estructura de datos auxiliar: Tabla de Memoización

La tabla de memoización es la estructura central de la PD. Puede ser un arreglo 1D, 2D o incluso un mapa hash. Transforma soluciones exponenciales en polinomiales al evitar recalcular subproblemas.

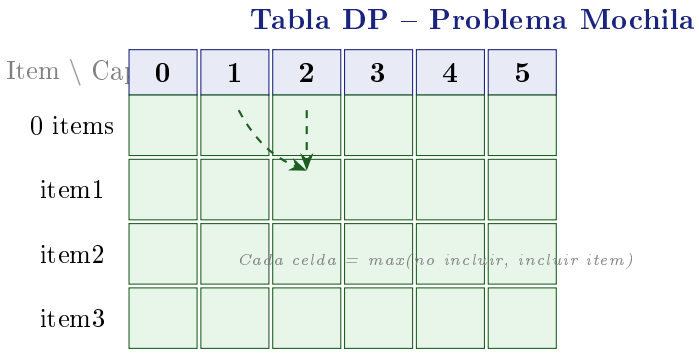

Figure 7: Tabla de DP para el problema de la Mochila 0/1. Cada celda $dp[i][c]$ almacena el valor máximo usando los primeros i ítems con capacidad c.

Listing 17: Fibonacci con memoización (Top-Down)

```java
// Sin PD: T(n) = T(n-1) + T(n-2) + 1 => O(2^n) -- exponencial
// Con PD: cada subproblema se calcula 1 sola vez => O(n)

public class Fibonacci {
    private static long[] memo;

    public static long fib(int n) {
        memo = new long[n + 1];
        Arrays.fill(memo, -1);
        return fibRec(n);
    }

    private static long fibRec(int n) {
        if (n <= 1)        return n;
        if (memo[n] != -1) return memo[n];  // ya calculado: O(1)
        memo[n] = fibRec(n-1) + fibRec(n-2); // calcular y guardar
        return memo[n];
    }

    // Bottom-Up (Tabulacion): mas eficiente, sin recursion
    public static long fibTabla(int n) {
        if (n <= 1) return n;
        long[] dp = new long[n + 1];
        dp[0] = 0; dp[1] = 1;
        for (int i = 2; i <= n; i++)
            dp[i] = dp[i-1] + dp[i-2];
        return dp[n];
    }
}
```

Listing 18: Problema de la Mochila 0/1 — PD Bottom-Up

```java
// items: {peso, valor}[]   capacidad: W
// dp[i][c] = max valor usando los primeros i items con cap. c
public static int mochila(int[] pesos, int[] valores, int W) {
    int n  = pesos.length;
    int[][] dp = new int[n + 1][W + 1];  // tabla de memoizacion 2D

    for (int i = 1; i <= n; i++) {
        for (int c = 0; c <= W; c++) {
```

```
            // Opcion 1: NO incluir el item i
            dp[i][c] = dp[i-1][c];

            // Opcion 2: incluir el item i (si cabe)
            if (pesos[i-1] <= c) {
                int conItem = valores[i-1] + dp[i-1][c - pesos
                    [i-1]];
                dp[i][c] = Math.max(dp[i][c], conItem);
            }
        }
    }
    return dp[n][W];
}

// Reconstruir cuales items se eligieron
public static List<Integer> reconstruir(int[] pesos, int[]
    valores,
                                            int[][] dp, int W) {
    List<Integer> elegidos = new ArrayList<>();
    int c = W;
    for (int i = dp.length-1; i >= 1; i--) {
        if (dp[i][c] != dp[i-1][c]) {  // este item fue
            incluido
            elegidos.add(i-1);
            c -= pesos[i-1];
        }
    }
    return elegidos;
}
// T(n,W) = O(nW)  -- pseudo-polinomial
// Espacio:          O(nW) en tabla, O(W) con optimizacion de
    una fila
```

Listing 19: Subsecuencia Común Más Larga (LCS)

```
// LCS("ABCBDAB", "BDCAB") = "BCAB" de longitud 4
// dp[i][j] = longitud de LCS entre s1[0..i-1] y s2[0..j-1]
public static int lcs(String s1, String s2) {
    int m = s1.length(), n = s2.length();
    int[][] dp = new int[m + 1][n + 1];

    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
```

```
            if (s1.charAt(i-1) == s2.charAt(j-1))
                dp[i][j] = 1 + dp[i-1][j-1];     // caracteres coinciden
            else
                dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);  // no coinciden
        }
    }
    return dp[m][n];
}
// T(n,m) = O(nm)    Espacio: O(nm)
```

Table 6: Comparativa: recursión vs. PD

Problema	Recursivo naive	Con PD	Mejora
Fibonacci(40)	$\approx 2^{40}$ llamadas	40 celdas	Exponencial $\to$ lineal
Mochila 0/1	2^n subconjuntos	$n \times W$ tabla	Exp $\to$ pseudo-polin.
LCS	2^{n+m}	$n \times m$ tabla	Exp $\to$ cuadrático
Camino en matriz	$O(2^n)$	$O(n^2)$	Exp $\to$ cuadrático

5.4 Algoritmos Voraces (*Greedy*)

Un **algoritmo voraz** toma en cada paso la decisión *localmente óptima* (la que parece mejor en ese momento) sin reconsiderar decisiones pasadas. No siempre da el óptimo global, pero cuando lo hace es muy eficiente.

Condiciones para que un Greedy sea óptimo

- **Propiedad greedy**: la elección local óptima lleva al óptimo global.
- **Subestructura óptima**: la solución óptima contiene soluciones óptimas de sus subproblemas.

Listing 20: Selección de actividades — Greedy clásico

```
// Dado un conjunto de actividades con inicio y fin,
// seleccionar el maximo numero de actividades no solapadas.
public static List<int[]> seleccionActividades(int[] inicio,
    int[] fin) {
    int n = inicio.length;
    Integer[] idx = new Integer[n];
```

```
    for (int i = 0; i < n; i++) idx[i] = i;

    // Ordenar por tiempo de finalizacion (criterio greedy)
    Arrays.sort(idx, (a, b) -> fin[a] - fin[b]);

    List<int[]> seleccionadas = new ArrayList<>();
    int ultimoFin = -1;

    for (int i : idx) {
        if (inicio[i] >= ultimoFin) {    // no se solapa con la ultima
            seleccionadas.add(new int[]{inicio[i], fin[i]});
            ultimoFin = fin[i];          // actualizar ultimo fin
        }
    }
    return seleccionadas;
}
// T(n) = O(n log n) por el ordenamiento
// Correcto: la prueba de intercambio garantiza que es optimo
```

Listing 21: Problema de la Mochila Fraccionaria — Greedy

```
// A diferencia de la mochila 0/1, aqui se pueden tomar fracciones.
// Estrategia greedy: tomar primero los items con mayor valor/peso.
public static double mochilafraccionaria(int[] pesos, int[] valores, int W) {
    int n = pesos.length;
    Double[] ratios = new Double[n];
    Integer[] idx   = new Integer[n];
    for (int i = 0; i < n; i++) {
        ratios[i] = (double) valores[i] / pesos[i]; // valor por unidad de peso
        idx[i]    = i;
    }
    // Ordenar por ratio descendente
    Arrays.sort(idx, (a, b) -> Double.compare(ratios[b], ratios[a]));

    double valorTotal   = 0;
    int    capacidadRestante = W;
```

```java

    for (int i : idx) {
        if (capacidadRestante == 0) break;
        int tomar = Math.min(pesos[i], capacidadRestante);
        valorTotal          += (double) tomar / pesos[i] *
            valores[i];
        capacidadRestante  -= tomar;
    }
    return valorTotal;
}
// T(n) = O(n log n)   -- OPTIMO garantizado para la version
    fraccionaria
// Nota: para Mochila 0/1 (enteros) el greedy NO garantiza
    optimo -> usar PD
```

Listing 22: Algoritmo de Huffman — Greedy + Cola de Prioridad

```java
// Construir el arbol de Huffman para comprension sin perdida
public class Huffman {

    // Nodo del arbol de Huffman
    static class Nodo implements Comparable<Nodo> {
        char  simbolo;
        int   frecuencia;
        Nodo  izq, der;
        Nodo(char s, int f)         { simbolo = s; frecuencia
            = f; }
        Nodo(int f, Nodo i, Nodo d) { frecuencia = f; izq = i;
             der = d; }
        boolean esHoja() { return izq == null && der == null;
            }
        public int compareTo(Nodo o) { return this.frecuencia
            - o.frecuencia; }
    }

    public static Nodo construir(char[] simbolos, int[] freqs)
        {
        // Insertar todas las hojas en la cola de min-
            prioridad
        PriorityQueue<Nodo> pq = new PriorityQueue<>();
        for (int i = 0; i < simbolos.length; i++)
            pq.add(new Nodo(simbolos[i], freqs[i]));

```

```java
        // Construir el arbol: combinar los dos nodos de menor frecuencia
        while (pq.size() > 1) {
            Nodo izq = pq.poll();  // menor frecuencia
            Nodo der = pq.poll();  // segunda menor
            pq.add(new Nodo(izq.frecuencia + der.frecuencia, izq, der));
        }
        return pq.poll();  // raiz del arbol
    }

    public static void imprimirCodigos(Nodo nodo, String codigo) {
        if (nodo.esHoja()) {
            System.out.println(nodo.simbolo + ": " + codigo);
            return;
        }
        imprimirCodigos(nodo.izq, codigo + "0");
        imprimirCodigos(nodo.der, codigo + "1");
    }
}
// T(n) = O(n log n)  donde n = numero de simbolos distintos
```

5.5 Comparativa de Técnicas

Table 7: Cuándo usar cada técnica de diseño

Técnica	Requisito	Ejemplo	Complejidad
Divide y Vencerás	Subproblemas independientes	MergeSort, FFT	$O(n \log n)$
Backtracking	Espacio de búsqueda acotado	N-Reinas, Sudoku	$O(b^d)$
B&B + Pruning	Función de cota disponible	TSP, mochila entera	Mejor que $O(b^d)$
Prog. Dinámica	Subprob. solapados + sub. ópt.	Mochila, LCS, caminos	$O(n^2)$–$O(nW)$
Algoritmo Voraz	Propiedad greedy demostrable	Mochila frac., Huffman	$O(n \log n)$

6 Resumen Integral y Guía de Decisión

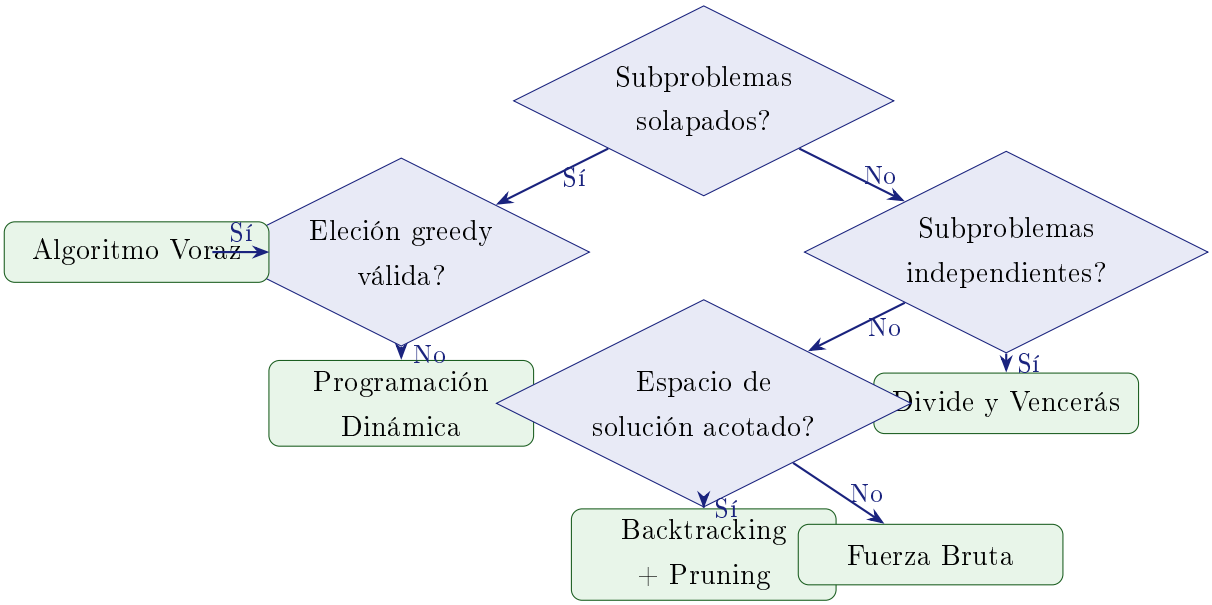

Figure 8: Diagrama de decisión: cómo elegir la técnica correcta.

Table 8: Resumen de complejidades de todos los algoritmos del curso

Algoritmo / Técnica	Mejor	Peor	Mem.
Bubble Sort	$O(n)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(1)$
Counting Sort	$O(n+k)$	$O(n+k)$	$O(k)$
Radix Sort	$O(nk)$	$O(nk)$	$O(n+k)$
MergeSort	$O(n \log n)$	$O(n \log n)$	$O(n)$
QuickSort	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Bús. Binaria	$O(1)$	$O(\log n)$	$O(1)$
Fibonacci (PD)	$O(n)$	$O(n)$	$O(n)$
Mochila 0/1 (PD)	$O(nW)$	$O(nW)$	$O(nW)$
LCS (PD)	$O(nm)$	$O(nm)$	$O(nm)$
Selección actividades	$O(n \log n)$	$O(n \log n)$	$O(n)$
Huffman	$O(n \log n)$	$O(n \log n)$	$O(n)$
N-Reinas (backtr.)	—	$O(n!)$	$O(n)$

7 Conclusiones

El análisis y diseño de algoritmos descansa sobre tres pilares:

1. **Medir correctamente**: la notación asintótica permite comparar algoritmos independientemente del hardware. Sumatorias y el método de los anuladores son las herramientas formales para derivar $T(n)$ de forma exacta.
2. **Ordenar eficientemente**: conocer las complejidades de los algoritmos de ordenamiento permite elegir el adecuado: Counting/Radix cuando los datos son enteros acotados; MergeSort cuando se necesita estabilidad y $O(n \log n)$ garantizado; QuickSort cuando la memoria es limitada y el promedio importa más que el peor caso.
3. **Diseñar estratégicamente**: ninguna técnica es universalmente superior. La clave es reconocer la estructura del problema: subproblemas solapados apuntan a PD, elección independiente a greedy, subproblemas separados a divide y vencerás, y búsqueda exhaustiva con restricciones a backtracking.

Próximos pasos recomendados

1. Resolver las recurrencias de cada algoritmo de ordenamiento con el método de los anuladores paso a paso.
2. Implementar todos los algoritmos desde cero, midiendo tiempo real con `System.nanoTime()`.
3. Estudiar el problema de la Mochila 0/1 con reconstrucción de solución óptima.
4. Explorar NP-Completitud: SAT, Vertex Cover, TSP.
5. Practicar en LeetCode/Codeforces los patrones de PD clásicos: caminos en matrices, partición de cadenas, juegos de dos jugadores.